```
# Запуск (в той же папке, что server.py):  python bot.py
import json, time, sqlite3, os
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup, WebAppIn
from telegram.ext import Application, CommandHandler, CallbackQueryHandler, Conte
import os
from server import db, gift, wadd, notify, raw_addr, ton_of, tadd, NANO, get_rid,

ADMINS = {int(x) for x in os.environ.get("ADMINS", "123456789").split(",") if x.s
URL = os.environ.get("APP_URL", "https://YOUR-DOMAIN/")      # https-адрес, где з
CHANNEL = os.environ.get("CHANNEL_URL", "https://t.me/")          # ссылка на ваш

def admin(f):
    async def w(u: Update, ctx: ContextTypes.DEFAULT_TYPE):
        if u.effective_user.id in ADMINS: await f(u, ctx)
    return w

def play_kb():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("🎰 Играть", web_app=WebAppInfo(url=URL))],
        [InlineKeyboardButton("📖 Как играть", callback_data="how"), InlineKeyboa

def welcome_text(name, known):
    if known:
        return f"👋 С возвращением, <b>{name}</b>!\n\nТвои номера и машины ждут в
    return (f"👋 Привет, <b>{name}</b>!\n\nДобро пожаловать в <b>UZ Номерок</b> 🚘
            "🎰 Крути номерные знаки Узбекистана\n💎 Ищи редкие и секретные номер
            "🚗 Выбивай машины и ставь на них номера\n🤝 Меняйся с другими игрока
            "🎁 Активируй промокоды\n\nНажми «Играть», чтобы начать!")

async def is_sub(bot, uid):  # подписан ли игрок на обязательный канал
    if not REQUIRED_CHANNEL: return True
    try:
        m = await bot.get_chat_member(REQUIRED_CHANNEL, uid)
        return m.status in ("member", "administrator", "creator")
    except Exception:
        return True   # не удалось проверить (бот не админ канала) - не блокируем

async def start(u: Update, ctx):
    user = u.effective_user; c = db()
    known = c.execute("SELECT 1 FROM users WHERE id=?", (user.id,)).fetchone()
    if ctx.args and ctx.args[0].startswith("ref_") and not known:   # ссылка-приг
        try: c.execute("INSERT OR REPLACE INTO refpending VALUES(?,?)", (user.id,
        except ValueError: pass
    name = user.first_name or "игрок"
```

```
    if not await is_sub(ctx.bot, user.id):
        kb = InlineKeyboardMarkup([[InlineKeyboardButton("📣 Подписаться", url=RE
                                   [InlineKeyboardButton("✅ Я подписался", callb
        return await u.message.reply_text(f"👋 Привет, <b>{name}</b>!\n\nЧтобы на
                                          reply_markup=kb, parse_mode="HTML")
    await u.message.reply_text(welcome_text(name, known), reply_markup=play_kb(),

async def chk(u: Update, ctx):
    q = u.callback_query; user = q.from_user
    if not await is_sub(ctx.bot, user.id):
        return await q.answer("Вы ещё не подписаны на канал", show_alert=True)
    await q.answer("Готово!")
    known = db().execute("SELECT 1 FROM users WHERE id=?", (user.id,)).fetchone()
    await q.message.reply_text(welcome_text(user.first_name or "игрок", known), r

async def precheckout(u: Update, ctx):
    await u.pre_checkout_query.answer(ok=True)

async def paid(u: Update, ctx):  # оплата звёздами прошла - выдаём подарок (вся л
    pay = u.message.successful_payment; d = json.loads(pay.invoice_payload)
    if int(d["u"]) != u.effective_user.id: return
    try: msg = apply_payment(db(), d, pay.total_amount, pay.telegram_payment_char
    except Exception as e:   # деньги получены, а выдача упала - не молчим: пишем
        import logging, traceback; logging.getLogger("uzn").error("ОПЛАТА НЕ ВЫДА
        await u.message.reply_text("⚠️ Оплата получена, но выдача не удалась. Мы
        for a in ADMINS:
            try: await ctx.bot.send_message(a, "🚨 Оплата не выдана: игрок %s, тс
            except Exception: pass
        return
    if msg: await u.message.reply_text(msg)
    try:   # уведомление админам: игрок заплатил звёздами
        c = db(); tu = u.effective_user; row = c.execute("SELECT name FROM users
        nm = (row["name"] if row and row["name"] else tu.full_name) or str(tu.id)
        tot = c.execute("SELECT COALESCE(SUM(stars),0) FROM payments WHERE charge
        txt = "💫 Донат звёздами!\n%s%s · ID %s\nТовар: %s\nСумма: %s ⭐\nВсего г
        for a in ADMINS:
            try: await ctx.bot.send_message(a, txt)
            except Exception as e: import logging; logging.getLogger("uzn").error
    except Exception as e:
        import logging; logging.getLogger("uzn").error("ошибка уведомления о дона

@admin
async def notifytest(u, ctx):  # /notifytest - проверка: придёт ли уведомление о
    ok = []; bad = []
    for a in ADMINS:
        try: await ctx.bot.send_message(a, "💫 Тест уведомления о донате: если вы
        except Exception as e: bad.append("%s (%s)" % (a, e))
```

```
    await u.message.reply_text("Админы из настроек (ADMINS): %s\nДоставлено: %s\n

@admin
async def stars(u, ctx):  # /stars - сколько звёзд получено
    c = db(); rows = c.execute("SELECT item, COUNT(*) n, SUM(stars) s FROM paymen
    tot = sum(r["s"] or 0 for r in rows); held = c.execute("SELECT COALESCE(SUM(s
    await u.message.reply_text(("\n".join(f'{r["item"]}: {r["n"]} шт · {r["s"]} ⭐

@admin
async def lastpay(u, ctx):  # /lastpay - последние платежи и сколько подарков ещё
    c = db(); rows = c.execute("SELECT charge,uid,item,stars,ts FROM payments WHE
    if not rows: await u.message.reply_text("Платежей пока нет: бот ещё ни разу н
    out = []
    for r in rows:
        w = c.execute("SELECT COUNT(*) FROM gifts WHERE uid=?", (r["uid"],)).fetc
        out.append("%s · ID %s · %s · %s ⭐ · подарков в очереди: %s" % (time.str
    await u.message.reply_text("Последние платежи (время Ташкента):\n" + "\n".joi

async def how(u: Update, ctx):
    await u.callback_query.answer()
    await u.callback_query.message.reply_text(
        "📖 <b>Как играть</b>\n\n1. Крути номера: Обычная, а с 5 и 7 уровня Улучш
        "2. Каждый номер можно продать или оставить в гараже.\n3. Повышай уровень
        "4. Машины выбиваются в рулетке «Машины», на них можно поставить номер.\n
        "5. Вкладка «Задания»: промокоды и обмен с игроками (нужен ID).", parse_m

SUPPORT = os.environ.get("SUPPORT", "@your_username")  # <- впиши свой @username

async def paysupport(u, c):
    await u.message.reply_text("По вопросам оплаты Stars пишите: " + SUPPORT)

async def terms(u, c):
    await u.message.reply_text("Покупки за Telegram Stars — виртуальные игровые п

async def post_init(app):
    await app.bot.set_my_commands([BotCommand("start", "Начать игру"), BotCommand
    await app.bot.set_chat_menu_button(menu_button=MenuButtonWebApp("🎰 Играть",
    await app.bot.set_my_short_description("Собирай номера и машины Узбекистана 🇺
    await app.bot.set_my_description("🎰 UZ Номерок: крути номерные знаки Узбекис

@admin
async def players(u, ctx):
    c = db()
    total = c.execute("SELECT COUNT(*) FROM users").fetchone()[0]
    rows = c.execute("SELECT id,name,state FROM users ORDER BY last_seen DESC LIM
    out = [f"Игроков: {total} (показаны последние 40)"]
    for r in rows:
```

```
        s = json.loads(r["state"] or "{}")
        out.append(f'{r["id"]} {r["name"]} · yp.{s.get("lv",1)} · {s.get("bal",0)
    await u.message.reply_text("\n".join(out))

def parse_gift(a):  # a = слова после ID игрока
    kind = a[0]
    if kind == "noads": return {"t": "noads"}
    if kind == "bal": return {"t": "bal", "v": int(a[1])}
    if kind == "plate": return {"t": "plate", "v": {"k": 0, "r": a[1], "a": a[2].
    if kind == "firm": return {"t": "plate", "v": {"k": 1, "r": a[1], "d": a[2],
    if kind == "car": return {"t": "car", "v": {"n": " ".join(a[2:]), "e": "🚗",
    if kind == "carplate":
        nm = " ".join(a[5:])
        return {"t": "car", "v": {"n": nm, "e": "🚙", "t": 3, "m": .8 if "9X" in
                "pl": {"k": 0, "r": a[1], "a": a[2].upper(), "d": a[3], "z": a[4]
    if kind == "boost":
        g = {"t": "boost", "k": {"luck": "bl", "sell": "bs", "xp": "bx"}[a[1]], "
        if len(a) > 3 and a[1] == "luck":   # множитель удачи: /give ID boost luc
            g["x"] = max(2, min(50, int(a[3])))
        return g
    raise ValueError

GIVE_HELP = ("Выдача: /give ID что-то (вместо ID можно all - всем игрокам)\n"
             "/give ID noads (отключить рекламу навсегда)\n/give ID bal 1000000\n
             "/give ID car 3 Название машины (0-4: обычная...секретная)\n/give ID
             "/give ID boost luck 120 (luck/sell/xp, минуты)\n/give ID boost luck

@admin
async def give(u, ctx):
    a = ctx.args
    try: g = parse_gift(a[1:]); target = a[0]
    except Exception: return await u.message.reply_text(GIVE_HELP)
    c = db()
    if target == "all":
        ids = [r["id"] for r in c.execute("SELECT id FROM users")]; who = f"всем
    else:
        row = c.execute("SELECT name FROM users WHERE id=?", (int(target),)).fetc
        if not row: return await u.message.reply_text("Игрок с таким ID не найден
        ids = [int(target)]; who = f"игроку {row['name']} ({target})"
    for i in ids: gift(c, i, g)
    c.commit()
    await u.message.reply_text(f"Выдано {who}. Игрок получит это при следующем за

@admin
async def addbal(u, ctx):  # /addbal ID СУММА - пополнить баланс игрока
    try: uid = int(ctx.args[0]); n = int(ctx.args[1])
    except Exception: return await u.message.reply_text("Использование: /addbal I
```

```
    c = db()
    if not c.execute("SELECT 1 FROM users WHERE id=?", (uid,)).fetchone(): return
    gift(c, uid, {"t": "bal", "v": n}); c.commit()
    await u.message.reply_text(f"Баланс игрока {uid} пополнен на {n:,} сум (придё

@admin
async def takebal(u, ctx):  # /takebal ID СУММА (или all) - списать деньги у игро
    try: uid = int(ctx.args[0]); w = ctx.args[1].lower().replace(" ", "")
    except Exception: return await u.message.reply_text("Использование: /takebal
    c = db(); row = c.execute("SELECT name,bal FROM users WHERE id=?", (uid,)).fe
    if not row: return await u.message.reply_text("Игрок с таким ID не найден")
    have = int(row["bal"] or 0)
    try: n = have if w in ("all", "все", "всё") else int(w)
    except ValueError: return await u.message.reply_text("Сумма должна быть число
    if n <= 0: return await u.message.reply_text("Сумма должна быть больше нуля")
    gift(c, uid, {"t": "bal", "v": -n}); c.commit()
    await u.message.reply_text(f"У игрока {row['name']} ({uid}) спишется {n:,} су

@admin
async def addstars(u, ctx):  # /addstars ID N - выдать (или списать, если N<0) зв
    try: uid = int(ctx.args[0]); n = int(ctx.args[1])
    except Exception: return await u.message.reply_text("Использование: /addstars
    c = db(); row = c.execute("SELECT name,COALESCE(stars,0) s FROM users WHERE i
    if not row: return await u.message.reply_text("Игрок с таким ID не найден")
    new = max(0, row["s"] + n); c.execute("UPDATE users SET stars=? WHERE id=?",
    c.execute("INSERT INTO wlog(uid,d,why,ts) VALUES(?,?,?,?)", (uid, new - row["
    await u.message.reply_text(f"Звёздный баланс игрока {row['name']} ({uid}): {r

@admin
async def broadcast(u, ctx):  # /broadcast текст - рассылка всем игрокам
    import asyncio
    parts = (u.message.text or "").split(None, 1)
    text = parts[1] if len(parts) > 1 else ""
    if not text: return await u.message.reply_text("Использование: /broadcast тек
    kb = InlineKeyboardMarkup([[InlineKeyboardButton("🎰 Играть", web_app=WebAppI
    ids = [r["id"] for r in db().execute("SELECT id FROM users")]
    ok = bad = 0
    for i in ids:
        try: await ctx.bot.send_message(i, text, reply_markup=kb); ok += 1
        except Exception: bad += 1
        await asyncio.sleep(0.06)
    await u.message.reply_text(f"Рассылка: доставлено {ok}, не доставлено {bad}")

@admin
async def uniq(u, ctx):  # /uniq - список единственных номеров; /uniq free ID - о
    c = db()
    if ctx.args[:1] == ["free"] and len(ctx.args) == 2:
```

```
        n = c.execute("DELETE FROM uniq WHERE owner=?", (int(ctx.args[1]),)).rowc
        return await u.message.reply_text(f"Освобождено единственных номеров: {n}
    rows = c.execute("SELECT u.key,u.owner,x.name FROM uniq u LEFT JOIN users x O
    tot = c.execute("SELECT COUNT(*) FROM uniq").fetchone()[0]
    def fmt(k):
        kk, r, a, d, z = k.split("|"); return f"{r} {a+' ' if a else ''}{d} {z}"
    await u.message.reply_text(f"Единственных номеров занято: {tot}\n" + "\n".joi
                               "\n\n/uniq free ID - освободить все номера игрока"

@admin
async def promo(u, ctx):
    a = ctx.args; c = db()
    if a and a[0] == "add" and len(a) == 4:      # /promo add CODE СУММА КОЛ-ВО
        c.execute("INSERT OR REPLACE INTO promos(code,reward,max_uses,used) VALUE
                  (a[1].upper(), json.dumps({"t": "bal", "v": int(a[2])}), int(a[
        await u.message.reply_text(f"Промокод {a[1].upper()} создан")
    elif a and a[0] == "del" and len(a) == 2:
        c.execute("DELETE FROM promos WHERE code=?", (a[1].upper(),)); c.commit()
    elif a and a[0] == "list":
        rows = c.execute("SELECT * FROM promos").fetchall()
        await u.message.reply_text("\n".join(f'{r["code"]}: {json.loads(r["reward
    else:
        await u.message.reply_text("/promo add КОД СУММА ЛИМИТ\n/promo list\n/pro

@admin
async def find(u, ctx):  # /find имя или ID - найти игрока и его ID
    q = " ".join(ctx.args).lower()
    if not q: return await u.message.reply_text("Использование: /find имя или ID"
    like = "%" + q.replace("\\", "\\\\").replace("%", "\\%").replace("_", "\\_")
    c = db(); c.create_function("pylower", 1, lambda x: (x or "").lower())  # LOW
    rows = c.execute("SELECT id,name,state FROM users WHERE CAST(id AS TEXT)=? OR
    out = []
    for r in rows:
        s = json.loads(r["state"] or "{}")
        out.append(f'{r["id"]} {r["name"]} · ур.{s.get("lv",1)} · {s.get("bal",0)
    await u.message.reply_text("\n".join(out) or "Не найдено. Использование: /fin

def _keep_plan(c):
    """Какие единственные номера оставить при сбросе: выпавшие в звёздной крутке
    import bisect
    pts = sorted(r[0] for r in c.execute("SELECT ts FROM payments WHERE item='spi
    own = {}; keys = set()
    for r in c.execute("SELECT key,owner,ts FROM uniq WHERE owner!=0"):
        own[r[0]] = (r[1], r[2]); parts = r[0].split("|")
        i = bisect.bisect_right(pts, r[2]) - 1                                  #
        star = i >= 0 and 0 <= r[2] - pts[i] <= 5
        if star or (len(parts) == 5 and parts[3] == "001" and parts[4] == "UZB"):
```

```
    try:   # номера, выигранные за ⭐ на аукционе, тоже остаются
        for r in c.execute("SELECT item FROM star_auc WHERE status='done' AND top
            try: keys.add(ukey(clean_plate(json.loads(r[0]))))
            except Exception: pass
    except Exception: pass
    plan = {}
    def take(uid, p, esc=False):
        if not isinstance(p, dict): return
        try: pp = clean_plate(p)
        except Exception: return
        k = ukey(pp); ow = own.get(k, (None,))[0]
        if k in keys and k not in plan and (ow == uid or (esc and ow in (-1, -3))
    for r in c.execute("SELECT id,state FROM users WHERE state IS NOT NULL"):
        try: st = json.loads(r[1])
        except Exception: continue
        for p in (st.get("col") or []): take(r[0], p)
        for car in (st.get("cars") or []):
            if isinstance(car, dict): take(r[0], car.get("pl"))
    for r in c.execute("SELECT uid,data FROM gifts"):
        try: g = json.loads(r[1])
        except Exception: continue
        if g.get("t") == "plate": take(r[0], g.get("v"))
    listed = set(); lkeys = set()
    for t, q in (("shop", "SELECT id,seller,item FROM shop WHERE status='open'"),
                 ("ton_lot", "SELECT id,seller,plate FROM ton_lot WHERE status='o
                 ("duels", "SELECT id,a,aplate FROM duels WHERE status='open'")):
        try: rows = c.execute(q).fetchall()
        except Exception: continue
        for lid, uid, it in rows:
            try: it = json.loads(it)
            except Exception: continue
            if not isinstance(it, dict): continue
            for cand in (it, it.get("pl"), it.get("v")):
                if not isinstance(cand, dict): continue
                try: k = ukey(clean_plate(cand))
                except Exception: continue
                if k in keys and own.get(k, (None,))[0] in (uid, -1, -3):
                    if t == "duels": take(uid, cand, True)
                    else: listed.add((t, lid)); lkeys.add(k)
                    break
    _keep_plan.listed = listed; _keep_plan.lkeys = lkeys
    return plan, sorted(keys - set(plan) - lkeys)

@admin
async def resetall(u, ctx):  # /resetall CONFIRM - сбросить всех, оставив номера
    full = ctx.args == ["CONFIRM", "ALL"]
    if ctx.args != ["CONFIRM"] and not full:
```

```
        c = db(); plan, lost = _keep_plan(c)
        lst = "\n".join("• %s — игрок %s" % (plate_txt(p), uid) for uid, p, _ in
        extra = ("\n…и ещё %d" % (len(plan) - 40)) if len(plan) > 40 else ""
        return await u.message.reply_text("⚠️ Сброс сотрёт у ВСЕХ игроков баланс,
                                          "Останутся: ID игроков, промокоды, плат
                                          ("\n\n⚠️ Не найдено в гаражах владельце
                                          "\n\nПеред сбросом делается копия базы.
    try: bk = backup_db("resetall")
    except Exception as e: return await u.message.reply_text(f"❌ Не удалось сдел
    c = db(); rid = get_rid(c) + 1; plan = {} if full else _keep_plan(c)[0]
    c.execute("CREATE TABLE IF NOT EXISTS claims(uid INT,id TEXT,PRIMARY KEY(uid,
    listed = set() if full else set(getattr(_keep_plan, "listed", ())); lkeys = s
    try:   # ставки на ⭐ аукционе возвращаются на звёздный баланс (у снимаемых л
        for a in c.execute("SELECT id,top,topuid FROM star_auc WHERE status='open
            if ("star_auc", a[0]) not in listed: wadd(c, a[2], a[1], "reset-refun
    except Exception: pass
    for t in ("shop", "star_auc", "ton_lot", "duels", "auctions"):
        try:
            for r in c.execute("SELECT id FROM " + t + " WHERE status='open'").fe
                if (t, r[0]) not in listed: c.execute("DELETE FROM " + t + " WHER
        except Exception: pass
    for lid in [x[1] for x in listed if x[0] == "auctions"]: c.execute("UPDATE au
    for q in ("UPDATE users SET state=NULL, bal=0", "DELETE FROM uniq WHERE (owne
              "DELETE FROM refpending", "DELETE FROM redeemed", "UPDATE promos SE
        c.execute(q, sorted(lkeys) if q.startswith("DELETE FROM uniq") else ())
    for k, (uid, p, ts) in plan.items():
        c.execute("INSERT OR REPLACE INTO uniq(key,owner,ts) VALUES(?,?,?)", (k,
    c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES('rid',?)", (str(rid),)); c
    await u.message.reply_text(f"✅ Прогресс всех игроков сброшен (сезон {rid}).

@admin
async def season(u, ctx):  # /season Название ДНИ - новый сезон (Battle Pass всех
    if len(ctx.args) < 2 or not ctx.args[-1].isdigit():
        c = db(); n = c.execute("SELECT v FROM meta WHERE k='season_name'").fetch
        left = max(0, int(e[0]) - int(time.time())) if e else 0
        return await u.message.reply_text(f"Сейчас сезон: {n[0] if n else 'Beta 2
    days = max(1, min(120, int(ctx.args[-1]))); name = " ".join(ctx.args[:-1])[:2
    c = db(); c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES('season_name',?)
    await u.message.reply_text(f"✅ Сезон «{name}» начался на {days} дн.")

@admin
async def stat(u, ctx):  # /stat - здоровье сервера и базы
    await u.message.reply_text(db_report())

@admin
async def backup(u, ctx):  # /backup - сделать копию базы и прислать файл сюда
    try: path = backup_db("manual")
```

```
    except Exception as e: return await u.message.reply_text(f"❌ Копия не создал
    size = os.path.getsize(path)
    if size > 49 * 1024 * 1024:
        return await u.message.reply_text(f"✅ Копия сохранена на сервере ({size
    with open(path, "rb") as f:
        await u.message.reply_document(f, filename=os.path.basename(path), captio

@admin
async def reset(u, ctx):  # /reset ID - сбросить прогресс одного игрока
    try: uid = int(ctx.args[0])
    except Exception: return await u.message.reply_text("Использование: /reset ID
    c = db()
    row = c.execute("SELECT name FROM users WHERE id=?", (uid,)).fetchone()
    if not row: return await u.message.reply_text("Игрок с таким ID не найден")
    c.execute("CREATE TABLE IF NOT EXISTS claims(uid INT,id TEXT,PRIMARY KEY(uid,
    c.execute("DELETE FROM uniq WHERE owner=?", (uid,))
    c.execute("DELETE FROM gifts WHERE uid=?", (uid,))
    c.execute("DELETE FROM offers WHERE frm=? OR to_id=?", (uid, uid))
    c.execute("DELETE FROM auctions WHERE seller=? AND status='open'", (uid,))
    c.execute("DELETE FROM claims WHERE uid=?", (uid,))
    c.execute("DELETE FROM redeemed WHERE uid=?", (uid,))
    c.execute("UPDATE users SET state=NULL WHERE id=?", (uid,))
    gift(c, uid, {"t": "reset"}); c.commit()
    await u.message.reply_text(f"✅ Прогресс игрока {row['name']} ({uid}) сброшен

@admin
async def event(u, ctx):
    a = ctx.args; c = db()
    if a and a[0] == "stop":
        c.execute("DELETE FROM events"); c.commit(); return await u.message.reply
    plain = [x for x in a if "=" not in x]
    if not plain or not plain[0].isdigit():
        return await u.message.reply_text("/event luck=3 sell=2 xp=2 upg=1 МИНУТЫ
    kv = {x.split("=")[0]: float(x.split("=")[1]) for x in a if "=" in x}
    c.execute("DELETE FROM events")
    c.execute("INSERT INTO events(name,luck,sell,xp,until,upg) VALUES(?,?,?,?,?,?
              (" ".join(plain[1:]) or "Ивент", kv.get("luck", 1), kv.get("sell",
    await u.message.reply_text("Ивент запущен")

@admin
async def nftlist(u, ctx):  # /nftlist - необработанные заявки на NFT
    rows = db().execute("SELECT * FROM nft WHERE status='pending' ORDER BY id LIM
    if not rows: return await u.message.reply_text("Заявок на NFT нет")
    await u.message.reply_text("\n".join("#%d  %s  игрок %s\n%s" % (r["id"], json

@admin
async def nftdone(u, ctx):  # /nftdone ID АДРЕС_NFT [ссылка] - NFT выпущен на кош
```

```
    try: nid = int(ctx.args[0]); rest = ctx.args[1:]
    except Exception: return await u.message.reply_text("Использование: /nftdone
    item = next((x for x in rest if raw_addr(x)), ""); link = next((x for x in re
    c = db(); r = c.execute("SELECT * FROM nft WHERE id=? AND status='pending'",
    if not r: return await u.message.reply_text("Заявка не найдена или уже обрабо
    c.execute("UPDATE nft SET status='minted',link=?,item=? WHERE id=?", (link, i
    notify(r["uid"], "💎 Ваш NFT выпущен и отправлен на кошелёк!" + ("\n" + link
    await u.message.reply_text("Готово, игрок уведомлён" + ("" if item else "\n⚠️

@admin
async def nftitem(u, ctx):  # /nftitem ID АДРЕС_NFT - дописать адрес NFT к уже вы
    try: nid = int(ctx.args[0]); item = ctx.args[1]
    except Exception: return await u.message.reply_text("Использование: /nftitem
    if not raw_addr(item): return await u.message.reply_text("Это не адрес TON")
    c = db(); c.execute("UPDATE nft SET item=? WHERE id=?", (item, nid)); c.commi

@admin
async def addton(u, ctx):  # /addton ID N - выдать TON на баланс (минус - забрать
    try: uid = int(ctx.args[0]); n = int(round(float(ctx.args[1]) * NANO))
    except Exception: return await u.message.reply_text("Использование: /addton I
    c = db()
    if not c.execute("SELECT 1 FROM users WHERE id=?", (uid,)).fetchone(): return
    if n < 0: n = -min(-n, ton_of(c, uid))
    tadd(c, uid, n, "admin"); c.commit(); await u.message.reply_text("Теперь у иг

@admin
async def tonwd(u, ctx):  # /tonwd - заявки на вывод TON
    rows = db().execute("SELECT * FROM ton_wd WHERE status='pending' ORDER BY id
    if not rows: return await u.message.reply_text("Заявок на вывод нет")
    await u.message.reply_text("\n\n".join("#%d  %.3f TON  игрок %s\n%s" % (r["id

@admin
async def tonpaid(u, ctx):  # /tonpaid ID - вы отправили TON игроку вручную
    try: i = int(ctx.args[0])
    except Exception: return await u.message.reply_text("Использование: /tonpaid
    c = db(); r = c.execute("SELECT * FROM ton_wd WHERE id=? AND status='pending'
    if not r: return await u.message.reply_text("Заявка не найдена или уже обрабо
    c.execute("UPDATE ton_wd SET status='paid' WHERE id=?", (i,)); c.commit(); no

@admin
async def tonno(u, ctx):  # /tonno ID - отклонить вывод, TON возвращаются на бала
    try: i = int(ctx.args[0])
    except Exception: return await u.message.reply_text("Использование: /tonno ID
    c = db(); r = c.execute("SELECT * FROM ton_wd WHERE id=? AND status='pending'
    if not r: return await u.message.reply_text("Заявка не найдена или уже обрабо
    c.execute("UPDATE ton_wd SET status='rejected' WHERE id=?", (i,)); tadd(c, r[
```

```
@admin
async def tonstat(u, ctx):  # /tonstat - сколько TON игроки держат на балансах (в
    c = db(); held = c.execute("SELECT COALESCE(SUM(ton),0) FROM users").fetchone
    dep = c.execute("SELECT COALESCE(SUM(amount),0) FROM ton_tx").fetchone()[0]
    await u.message.reply_text("Принято депозитов: %.3f TON\nНа балансах игроков:

@admin
async def nftno(u, ctx):  # /nftno ID - отклонить заявку, номер возвращается игро
    try: nid = int(ctx.args[0])
    except Exception: return await u.message.reply_text("Использование: /nftno ID
    c = db(); r = c.execute("SELECT * FROM nft WHERE id=? AND status='pending'",
    if not r: return await u.message.reply_text("Заявка не найдена или уже обрабо
    p = json.loads(r["plate"]); c.execute("UPDATE nft SET status='rejected' WHERE
    c.execute("UPDATE uniq SET owner=? WHERE key=? AND owner=-2", (r["uid"], r["k
    notify(r["uid"], "Заявка на NFT отклонена, номер вернулся в гараж.")
    await u.message.reply_text("Отклонено, номер возвращён")

@admin
async def ultra(u, ctx):  # /ultra 1 (Аюб) или /ultra 2 (Бибиша)
    try: i = int(ctx.args[0])
    except Exception: i = 0
    await u.message.reply_text(launch_ultra(db(), i))

@admin
async def ads(u, ctx):  # /ads off - выключить рекламу всем, /ads on - включить
    a = (ctx.args[0].lower() if ctx.args else "")
    c = db()
    if a in ("off", "on"):
        c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES('ads_off',?)", ("1" if
    await u.message.reply_text("Реклама сейчас: %s\n/ads off - выключить всем\n/a

@admin
async def adsint(u, ctx):  # /adsint 60 или /adsint 60 10 (секунды между рекламой
    c = db()
    try:
        i = int(ctx.args[0]); f = int(ctx.args[1]) if len(ctx.args) > 1 else ads_
        if not (20 <= i <= 3600 and 1 <= f <= 100): raise ValueError
        c.execute("INSERT OR REPLACE INTO meta(k,v) VALUES('ads_cfg',?)", ("%d,%d
    except IndexError: pass
    except ValueError: return await u.message.reply_text("Секунды от 20 до 3600,
    i, f = ads_cfg(c)
    await u.message.reply_text("Реклама: раз в %d сек, до %d штук подряд, потом п

@admin
async def newseason(u, ctx):   # /newseason Beta 3 14 - показать итоги и план; /n
    a = list(ctx.args); conf = bool(a) and a[-1] == "CONFIRM"
    if conf: a = a[:-1]
```

```
    if len(a) < 2 or not a[-1].isdigit():
        return await u.message.reply_text("Новый сезон: /newseason Название ДНИ\n
                                          "Что произойдёт: у ВСЕХ сумы станут %s
    days = max(1, min(120, int(a[-1]))); name = " ".join(a[:-1])[:24]
    c = db(); old, P = season_plan(c)
    tw = sorted([i for i in P if P[i]["wr"]], key=lambda i: P[i]["wr"])[:10]; tl
    act = sum(1 for x in P.values() if x["act"]); part = sum(1 for x in P.values(
    tot_m = sum(x["rew"].get("m", 0) for x in P.values()); tot_s = sum(x["rew"].g
    txt = "Сезон «%s» → «%s» на %d дн.\nИгроков: %d, играли в сезоне: %d\n\n🏁 Тс
        old, name, days, len(P), act,
        "\n".join("%d. %s — %d побед из %d" % (P[i]["wr"], P[i]["name"], P[i]["w"
        "\n".join("%d. %s — уровень %d" % (P[i]["lr"], P[i]["name"], P[i]["lv"])
        part, f"{tot_m:,}".replace(",", " "), tot_s, f"{SEASON_BAL:,}".replace(",
    if not conf:
        return await u.message.reply_text(txt + "\n\nЭто только план, ничего не и
    try: bk = backup_db("newseason")
    except Exception as e: return await u.message.reply_text(f"❌ Не удалось сдел
    season_apply(c, P, old, name, days)
    await u.message.reply_text("✅ Сезон «%s» начался на %d дн.\n\n%s\n\nИгроки п
        name, days, txt, name, f"{SEASON_BAL:,}".replace(",", " ")))

async def mute(u, ctx):  # /mute - отключить рассылки игры
    c = db(); c.execute("UPDATE users SET mute=1 WHERE id=?", (u.effective_user.i

async def unmute(u, ctx):
    c = db(); c.execute("UPDATE users SET mute=0 WHERE id=?", (u.effective_user.i

app = Application.builder().token(BOT_TOKEN).post_init(post_init).build()
app.add_handler(CallbackQueryHandler(how, pattern="^how$"))
app.add_handler(CallbackQueryHandler(chk, pattern="^chk$"))
app.add_handler(PreCheckoutQueryHandler(precheckout))
app.add_handler(MessageHandler(filters.SUCCESSFUL_PAYMENT, paid))
for n, f in [("start", start), ("players", players), ("give", give), ("ultra", ul
    app.add_handler(CommandHandler(n, f))
app.run_polling()
```